Servo Motor Control Design Activity Sheet

Student Name: Alex Ong

Student Number: n12435562

Preparations for the task:

The next code helps prepare your MATLAB work space (you don't need to edit, unless error occurs)

```
%The next 3 lines change MATLAB's current working folder to this .mlx script's
current location.
filePath = matlab.desktop.editor.getActiveFilename;
filePath = fileparts(filePath);
cd(filePath)
%If causes a MATLAB error on your computer, remove these 3 code lines and manually
change folder.

clear all
close all
```

Overview

In this activity you will work with the **input to shaft angle position** servo motor transfer function:

$$G(s) = \frac{\Theta(s)}{V_m(s)} = \frac{K_m}{s(s+\alpha)}$$

and investigate (unity) negative feedback for shaft angle position control based on shaft angle measurements. Here V_m is input signal and Θ is output shaft position.

Assessed Tasks – Control System Design

*In the following, you will perform a control system design for the "***input to shaft angle position servo motor"** ***with the parameter values you examined in the EGB345 Servo Motor System Identification task.***

Preliminary information: your servo motor parameter values for this activity.

In the below tasks use for your servo motor parameter values, and *a,* the following:

- The K_m and α values provided in Task 1.1 of the EGB345 Servo Motor System Identification task.

In this activity we investigate (unity) negative feedback closed-loop system, where the "unity" means consider H=1 on the feedback path.

Task 1: Design feedback control for a requested response

Aim: Build a MATLAB representation of your servo motor system, design feedback control and simulate.

In the MATLAB for this task use the following MATLAB variables in your MATLAB:

km, alpha, KTask1.

Complete the following sub-tasks:

Task 1.1:

Build a MATLAB representation of the **input to shaft angle position** servo motor in open loop (i.e. no controller and no feedback) having your servo motor parameter values. We call this representation *your open-loop system*. Analytically assess the expected unit step response of your open loop system. Plot the unit step response of your open-loop system. Comment on whether the plot matches your analytic assessment.

ANSWER (edit below code block)

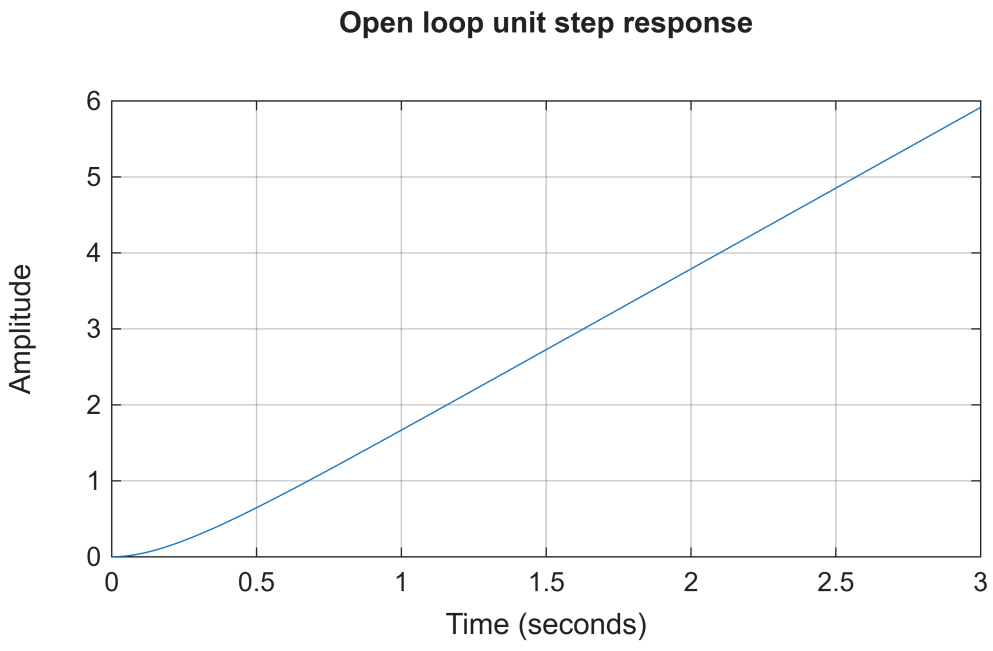
```
km = 9.76
```

```
km =
9.7600
```

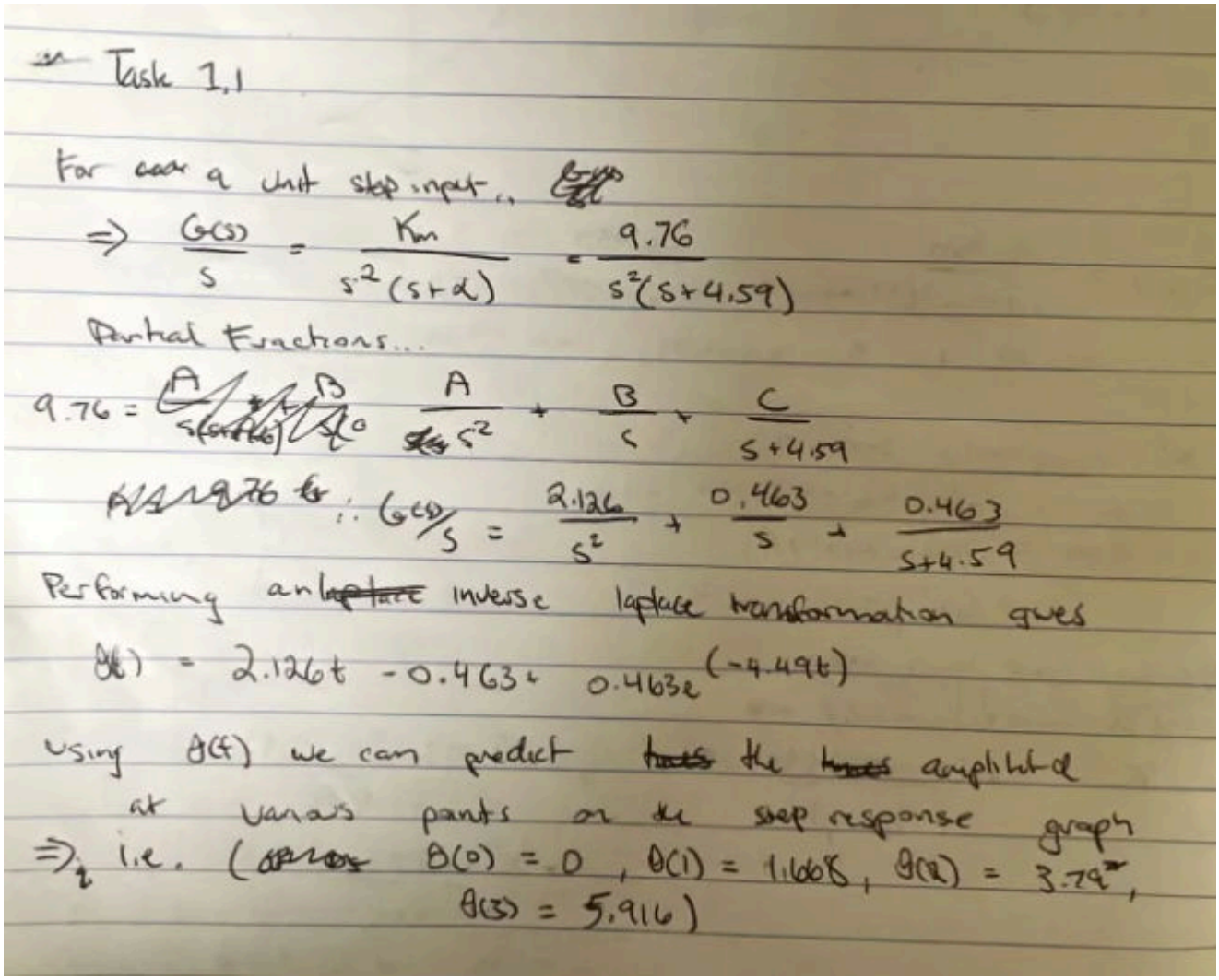
```
alpha = 4.59
```

```
alpha =
4.5900
```

```
s = tf('s');
G = km/(s*(s+alpha));
figure; step(G, 3); grid on
title('Open loop unit step response')
```



ANSWER (edit below text section)

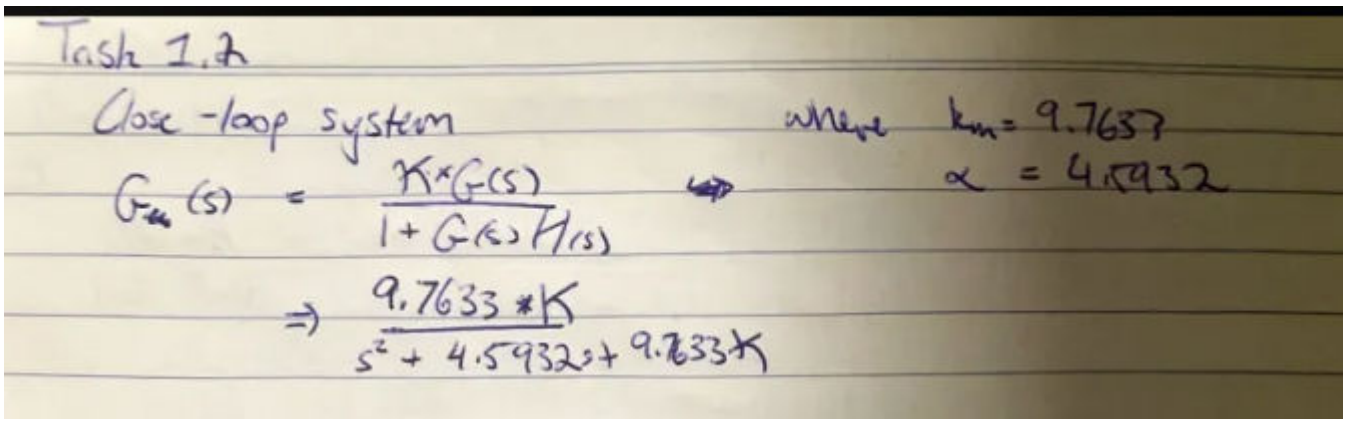


Analytically, the numbers matches the step response given by the graph as the visually match with each other.

Task 1.2:

Analytically create the transfer function expression of a (unity) negative feedback closed-loop system containing a general gain K controller in front of your open-loop system.

ANSWER (edit below text section)



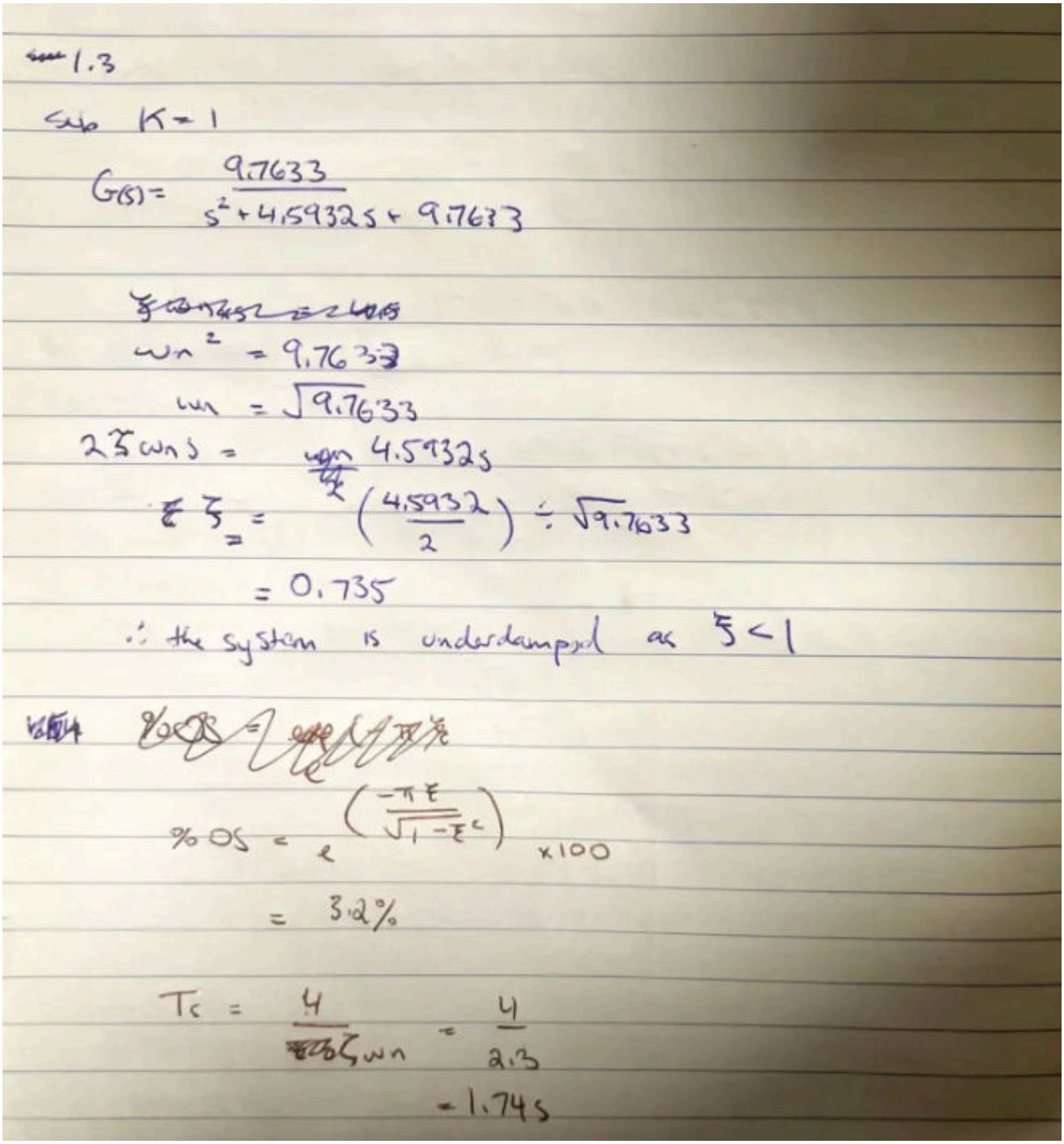
Task 1.3:

For *K=1*, if underdamped in closed-loop, analytically determine the settling time and % overshoot. Note we don't have an analytic formula for over-damped systems, and the formula is not accurate if the system is only slightly underdamped. Hence, if overdamped, or if only slightly underdamped, in closed-loop then use a MATLAB generated unit step response plot to graphically estimate settling time (after completing Task 1.5 below).

ANSWER (if needed, edit below code block)

```
% Only need if your system is overdamped, or close to overdamped.
% since the system is underdamped this is unecessary
```

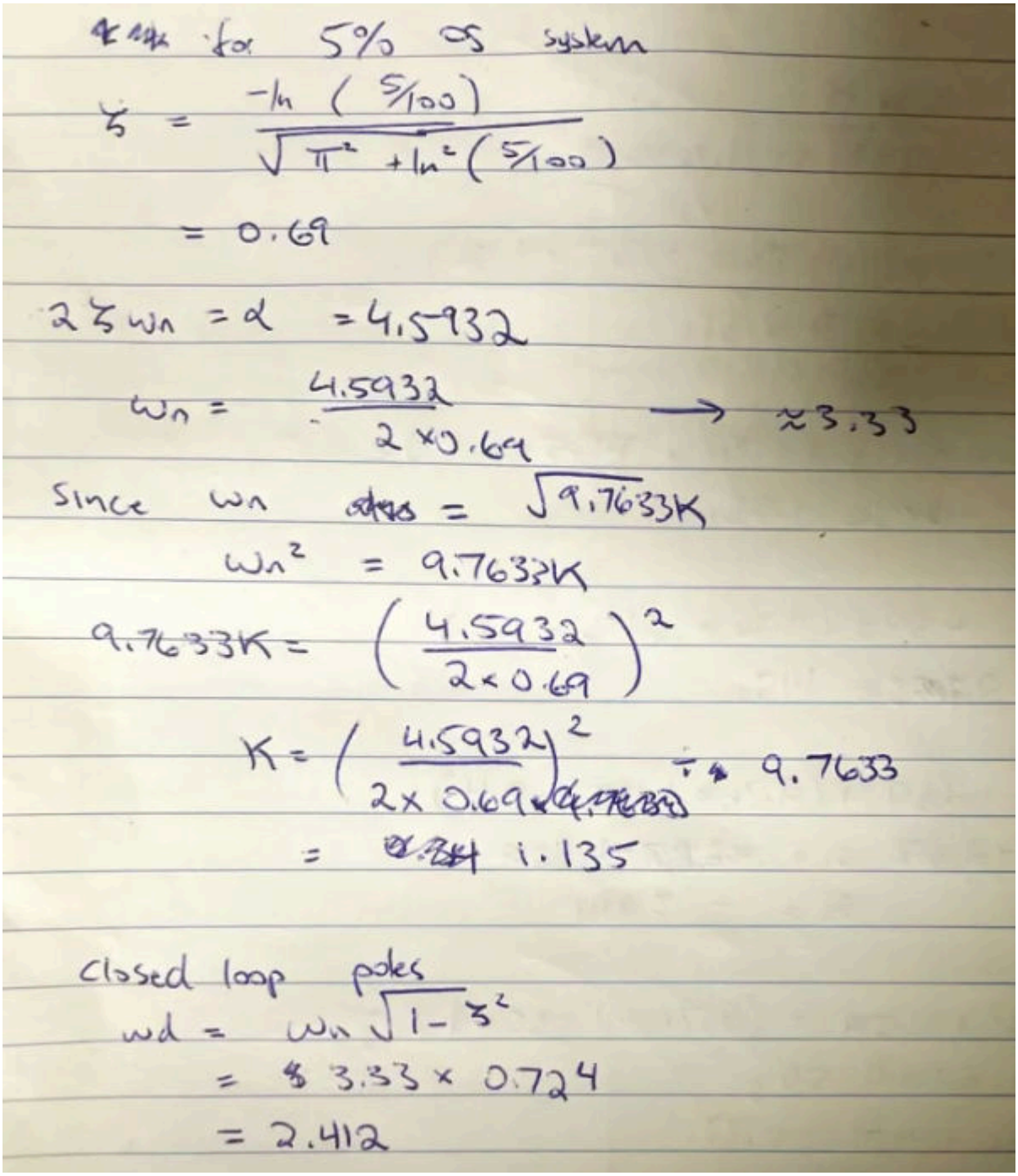
ANSWER (edit below text section)



Task 1.4:

Using mathematical analysis, determine a value of *K* that results in a 5% overshoot. Report the location of the closed-loop poles that would result. Label this designed gain *KTask1* (i.e. K from task 1).

ANSWER (edit below text section)



Ktask1 = 1.135

Task 1.5:

Build an appropriate MATLAB representation of the closed-loop system for *K=1*. Plot the unit step response of the closed-loop system.

Your MATLAB code:

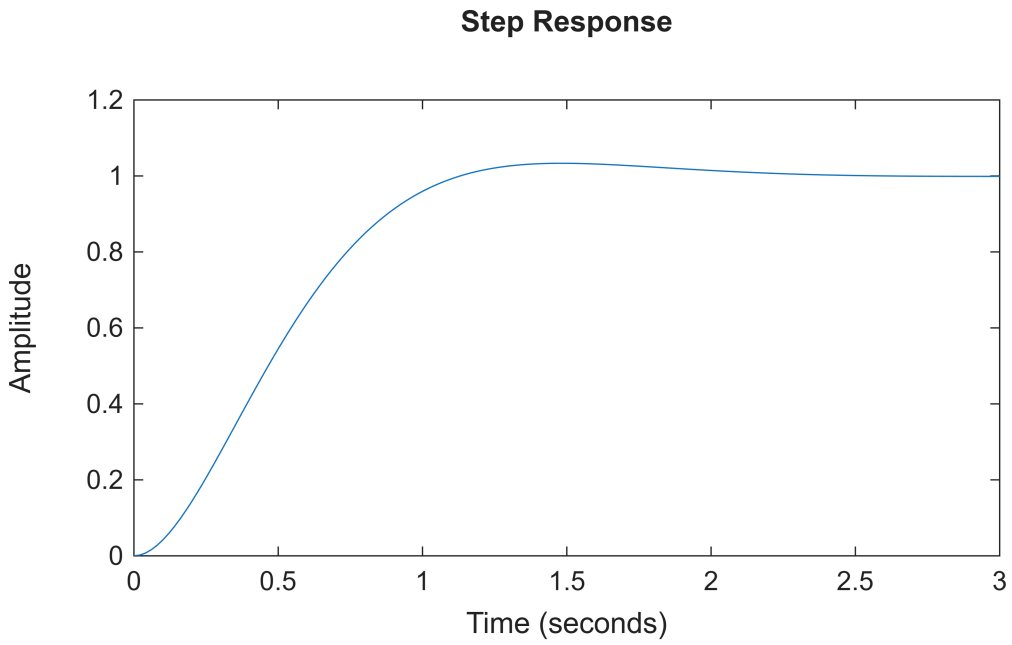
```
km = 9.76
```

```
km =
9.7600
```

```
alpha = 4.59
```

```
alpha =
4.5900
```

```
Numerator = km;
Denominator = [1 alpha 0];
Gs = tf(Numerator, Denominator);
Hs = 1;
Gcl = (Gs / (1 + Gs * Hs) );
stepplot(Gcl);
```




```
stepinfo(Gcl)
```

```
ans = struct with fields:
         RiseTime: 0.7159
    TransientTime: 1.8736
     SettlingTime: 1.8736
      SettlingMin: 0.9027
      SettlingMax: 1.0333
        Overshoot: 3.3323
       Undershoot: 0
             Peak: 1.0333
         PeakTime: 1.4849
```

Task 1.6:

Build an appropriate MATLAB representation of the closed-loop system with the designed value of *K* for 5% overshoot (i.e. *KTask1)*. Plot the unit step response of the closed-loop system.

Your MATLAB code:

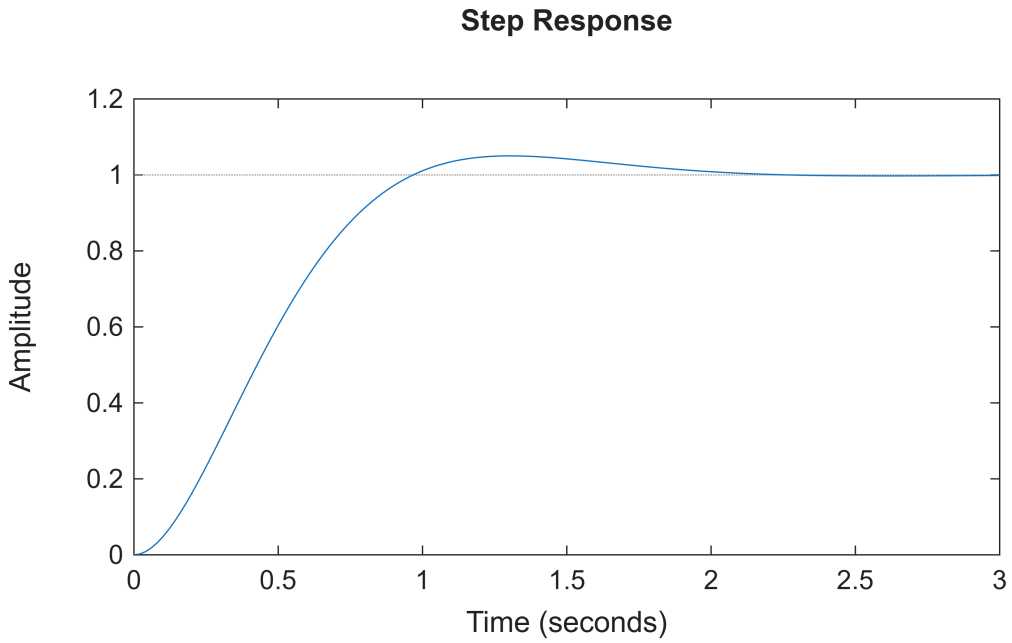
```
K = 1.135;
km = 9.76
```

```
km =
9.7600
```

```
alpha = 4.59
```

```
alpha =
4.5900
```

```
Numerator = km;
Denominator = [1 alpha 0];
Gs = tf(Numerator, Denominator);
Hs = 1;
Gcl = feedback(K*Gs, Hs);
stepplot(Gcl);
```




```
stepinfo(Gcl)
```

```
ans = struct with fields:
         RiseTime: 0.6296
    TransientTime: 1.8016
     SettlingTime: 1.8016
      SettlingMin: 0.9013
      SettlingMax: 1.0502
        Overshoot: 5.0235
       Undershoot: 0
             Peak: 1.0502
         PeakTime: 1.3043
```

Task 1.7:

Comment on whether the 3 unit step responses seen in the previous tasks (Task 1.1, 1.5 and 1.6) match your analytic results (quantitative and qualitative assessment).

ANSWER (edit below text section)

- Overshoot is similar in both cases.
- Simulation settling time is slightly longer due to the 2nd-order approximation.
- Both final values are 1, as predicted by T(0)=1
- Feedback makes the ramp response stable.
- Increasing K from 1 to 1.145 raises overshoot from 3% to 5% as daping ratio drops from 0.74 to 0.69.
- Both systems have poles with real part -2.3, so settling times are similar.
- The plant's integrator gives zero steady-state error to a step.

Here and below, the word “analytically” means the outcome of mathematical analysis, see Learning Requirements and Expectations module on CANVAS for more explanation. We would expect several mathematical operations deriving the required mathematical object.

Task 2:Investigation of the gain K

Aim: Evaluate the impact of errors in the value of designed K (i.e. *KTask1*), such as arises due to imperfect hardware components.

Complete the following sub-tasks:

Task 2.1:

Analytically determine the location of the (unity) negative feedback closed-loop poles for at least 5 values of *K* below and at least 5 values above the K value you designed in Task 1 (i.e. *KTask1*). Ensure a selection of *K* values so that the closed-loop system shows examples of both over and underdamped closed-loop unit step response behaviour.

We initially suggest increasing and decreasing by a factor of 20% (that is 0.8**KTask1*, 0.6* *KTask1*, … and 1.2* *KTask1* etc., but avoid using 0* *KTask1*). Ensure you select a range of *K* values showing examples for both over and underdamped closed-loop unit step response behaviour in closed-loop.

Identify one underdamped pole. Identify one overdamped pole. Explain the meaning and significance of your choices.

ANSWER (edit below text section)

Task 2.1

$\alpha = 4.5932$

$K = 1.135$

$$C_{CO} = \frac{9.76 K}{s^2 + 4.5932 s + 9.76 K}$$

1.2 x K: $s^2 + 4.59 + (9.76 \times 1.135 \times 1.2)$

poles $\Rightarrow$ $-2.3 \pm j 2.81$

0.8 x K: $s^2 + 4.59 + (9.76 \times 1.135 \times 0.8)$

poles $\Rightarrow$ $-2.3 \pm j 1.88$

0.6 x K: $s^2 + 4.59 + (9.76 \times 1.135 \times 0.6)$

poles $\Rightarrow$ $-2.3 \pm j 1.15$

0.4 x K: $s^2 + 4.59 + (9.76 \times 1.135 \times 0.4)$

poles $\Rightarrow$ $s_1 = -1.359$

$s_2 = -3.241$

0.2 x K: $s^2 + 4.59 + (9.76 \times 1.135 \times 0.2)$

poles $\Rightarrow$ $s_1 = -0.543$

$s_2 = -4.057$

0.3 x K: $s^2 + 4.59 + (9.76 \times 1.135 \times 0.3)$

poles $\Rightarrow$ $s_1 = -0.901$

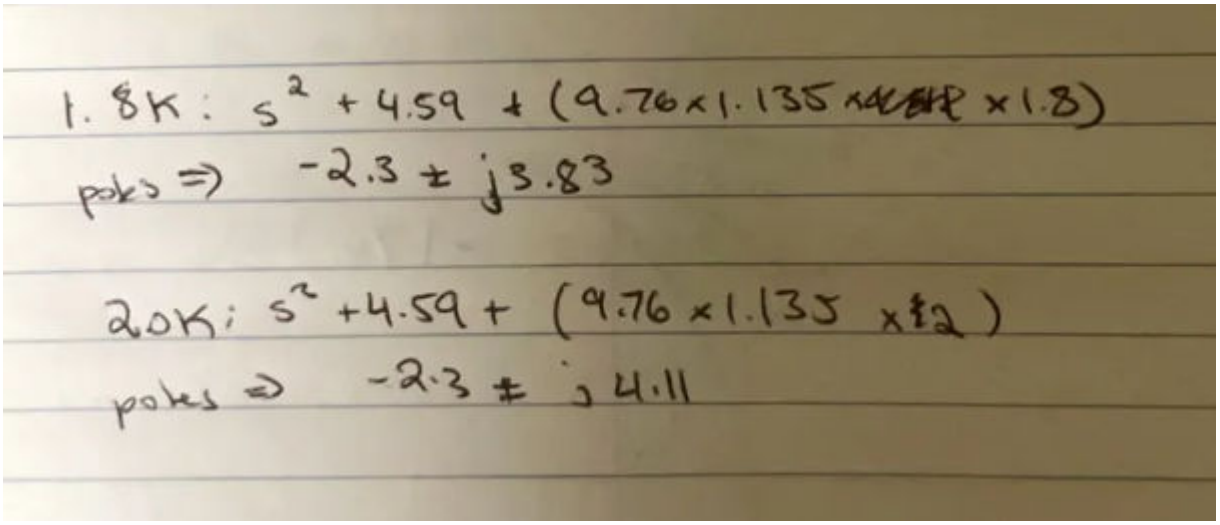
$s_2 = -3.689$

1.4 K: $s^2 + 4.59 + (9.76 \times 1.135 \times 1.4)$

poles $\Rightarrow$ $-2.3 \pm j 3.2$

1.6 K: $s^2 + 4.59 + (9.76 \times 1.135 \times 1.6)$

poles $\Rightarrow$ $-2.3 \pm j 3.529$



Overdamped (0.2K): poles −0.548 and −4.042 are both real, ζ = α/(2√(Km·K)) = 1.54 > 1. There is no overshoot, so the time constant is about 1.8 s and settling takes about 7 s.

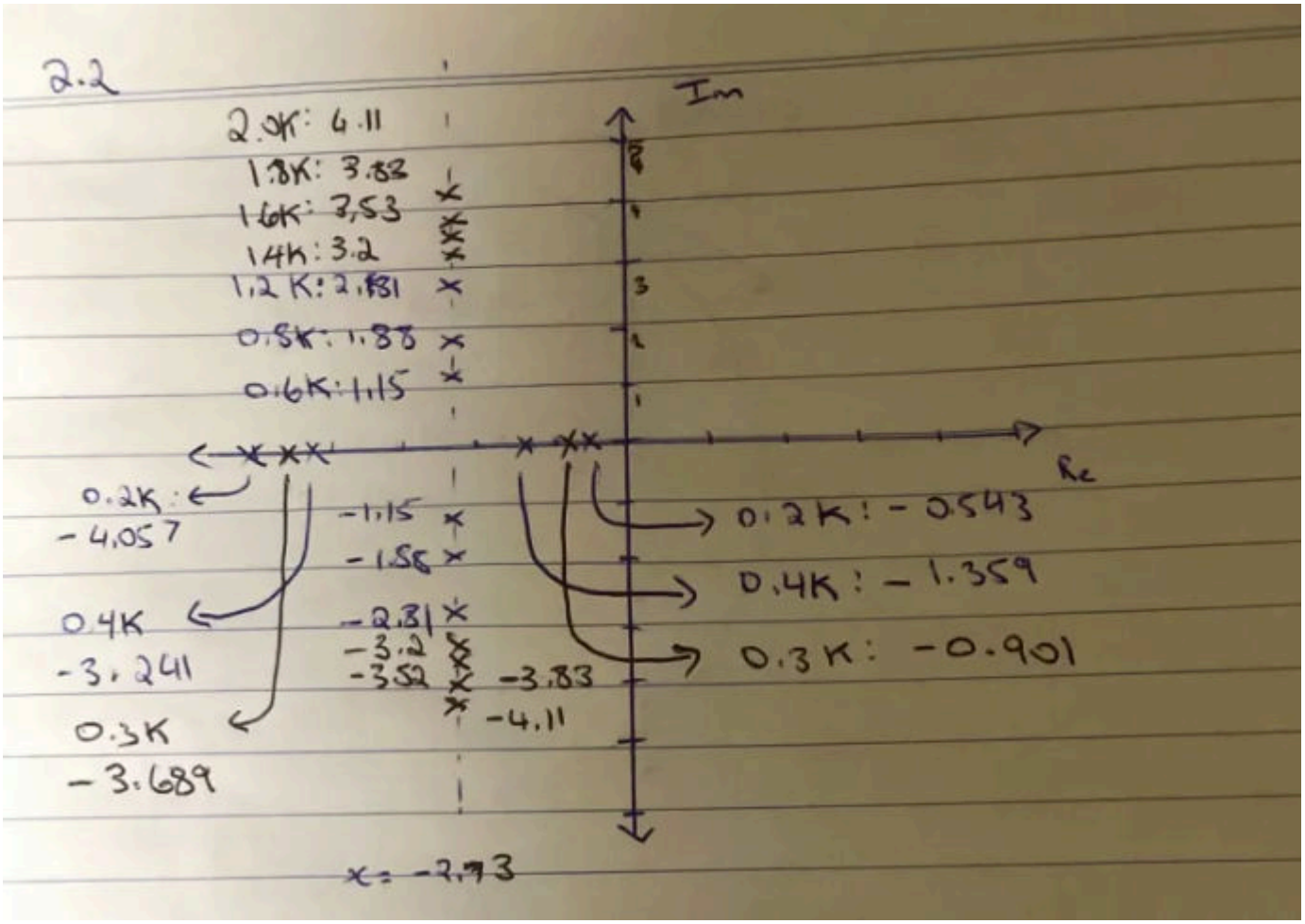
Underdamped (1.2K): poles −2.295 ± j2.833, ζ = 0.63 < 1. The overshoot is about 7.8 %

Too little gain makes the response sluggish.

Task 2.2:

Neatly hand draw a graph in the s-plane that reports the closed-loop pole locations used in the previous sub-task (on paper or electronic equivalent but MATLAB-generated plots are not accepted for this specific sub-task). Neatly label a couple of the poles with their corresponding *K* value. Comment on the meaning of what you see in the graph you have drawn (quantitative and qualitative assessment). Provide connections to concepts encountered in lectures (qualitative).

ANSWER (edit below text section)



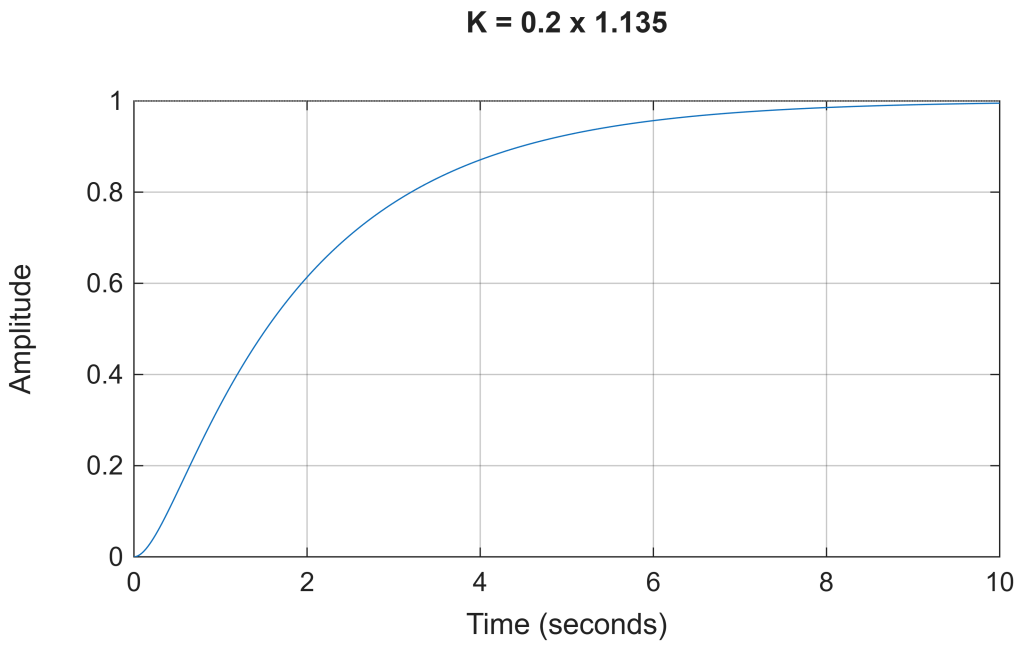
Task 2.3:

Using your MATLAB representation of the closed-loop system, plot the unit step response of the following 3 cases: the designed *K (i.e. KTask1)*, and the largest and smallest values of *K* you tested in the above sub-tasks. Comment on what you see in the unit step response plots (quantitative and qualitative assessment).

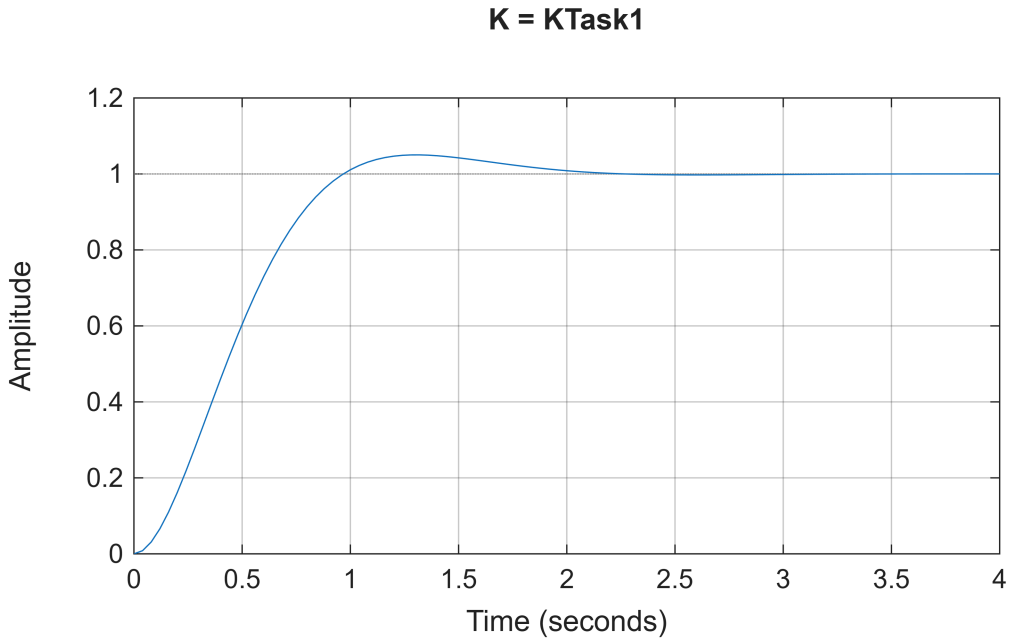
ANSWER (edit below code block)

```
KTask1 = 1.135;
K1 = 0.2*KTask1;
K2 = KTask1;
K3 = 1.2*KTask1;
T1 = feedback(K1*G, 1);
T2 = feedback(K2*G, 1);
T3 = feedback(K3*G, 1);

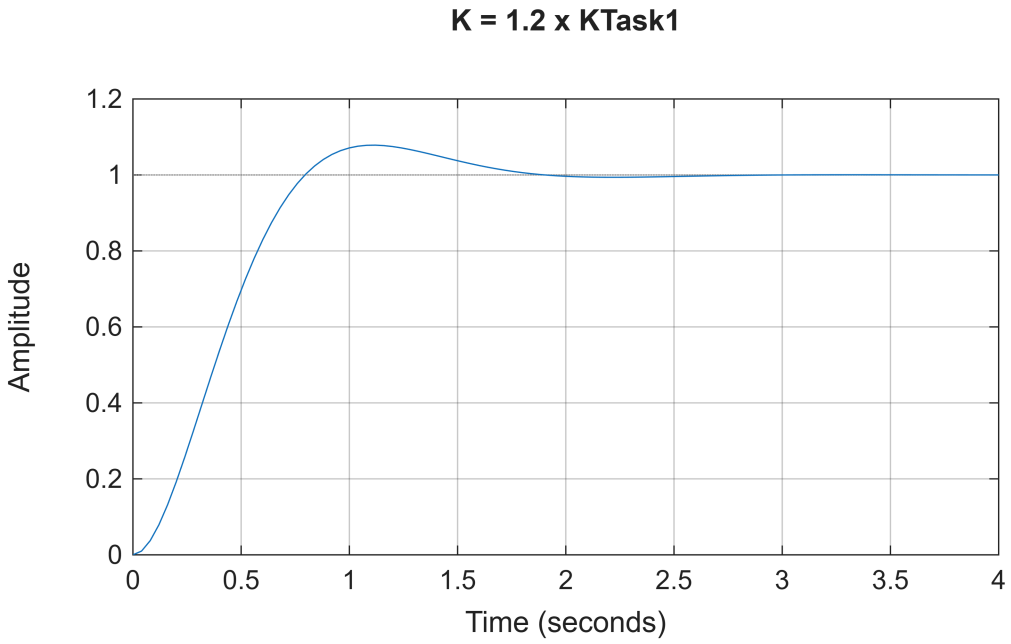
figure
step(T1, 10); grid on
title('K = 0.2 x 1.135')
saveas(gcf, "twofigure1.png");
```




```
figure
step(T2, 4); grid on
title('K = KTask1')
saveas(gcf, "twofigure2.png");
```




```
figure
step(T3, 4); grid on
title('K = 1.2 x KTask1')
saveas(gcf, "twofigure3.png");
```



ANSWER (edit below text section)

- Lowest K has no overshoot but takes 4 times longer to settle.
- Designed K gives 5% overshoot.
- Doubling K gives 17% overshoot, with similar settling time (1.73 s vs 1.80 s).
- Low K: smooth but slow.
- High K: faster rise and more oscillation.
- All settle at 1 due to the plant's integrator.
- A ±20% error in K changes overshoot from 5% to about 2% or 8%.

Task 3: Robustness study - an extra unmodelled fast pole

Aim: Investigate the impact of an extra unmodelled fast pole.

You are told the true system has an additional fast pole at β and is described by:

$$G_{extra}(s) = \frac{\Theta(s)}{V_m(s)} = \frac{\beta K_m}{s(s+\alpha)(s+\beta)}$$

You are told the unmodelled pole at -β is faster than the pole at -α but you don't know how much faster ("unmodelled" means you didn't know the pole was present and/or didn't know its specific value). Note the extra factor in the numerator ensures the systems with different extra poles have the same final value for their unit step response (in open-loop).

You do not need to consider the gain error issue considered in Task 2.

Consider the three systems:

- No extra pole (same as used in Task 1 and elsewhere):

$$G_{ne}(s) = \frac{\Theta(s)}{V_m(s)} = \frac{K_m}{s(s+\alpha)}$$

- Fast extra pole where $\beta = 10\alpha$

$$G_{fo}(s) = \frac{\Theta(s)}{V_m(s)} = \frac{10\alpha K_m}{s(s+\alpha)(s+10\alpha)}$$

- Slow extra pole where $\beta = 2\alpha$

$$G_{so}(s) = \frac{\Theta(s)}{V_m(s)} = \frac{2\alpha K_m}{s(s+\alpha)(s+2\alpha)}$$

In Task 1 you previously designed a gain *K* for (unity) negative feedback closed loop system to achieve a 5% overshoot assuming no extra pole system. Reused this previously designed gain *KTask1* (i.e. K designed from sub-task 1.6).

In the MATLAB for this task use the following MATLAB variables in your MATLAB:

km, alpha, KTask1, Kfo, Kso,

In this task you must use the root locus graphical design method taught in the last few lectures of EGB345.

Complete the following sub-tasks:

Task 3.1:

Complete the hand graphical and analytic steps to design a new controller gain *K* (for (unity) negative feedback closed loop system) to achieve 5% overshoot assuming that $G_{fo}(s)$ is the true model. You should hand draw the root locus. Label this designed gain *Kfo*.

ANSWER (edit below text section)

No. asymptotes = 5 - 0 = 3

COA = (0 - 4.6 - 4.6)/3 = -16.67

AOA = 180°(2+1)/3

= 60°, 180°, 300°

Since 5% $\zeta = 0.69$

$\cos(\theta) = \zeta = 0.69$

$\theta = \cos^{-1}(0.69)$

$= 46.37°$

Drawing the 46.37° line gives an intersection at

$s = -2.24 + j\,2.3$

$G_{fb}(s) = \frac{10\alpha K_m}{s(s+\alpha)(s+10\alpha)}$

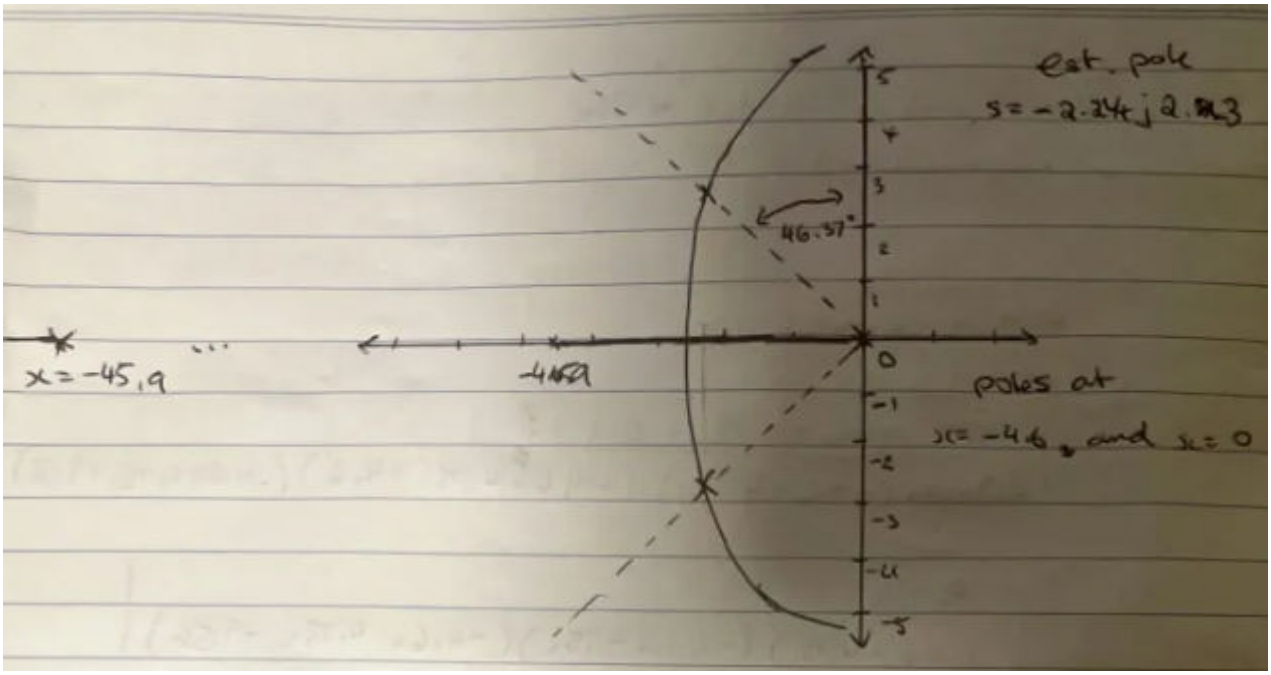
since $\alpha = 4.59$ poles @:

$x = 0,\ x = -4.59,\ x = -45.9$

$$K = -\frac{s(s+4.59)(s+45.9)}{448}$$

$$= \frac{-s^3 + 50.649s^2 + 210.68s}{448}$$

$s = -2.24$ or $s = -31.42$

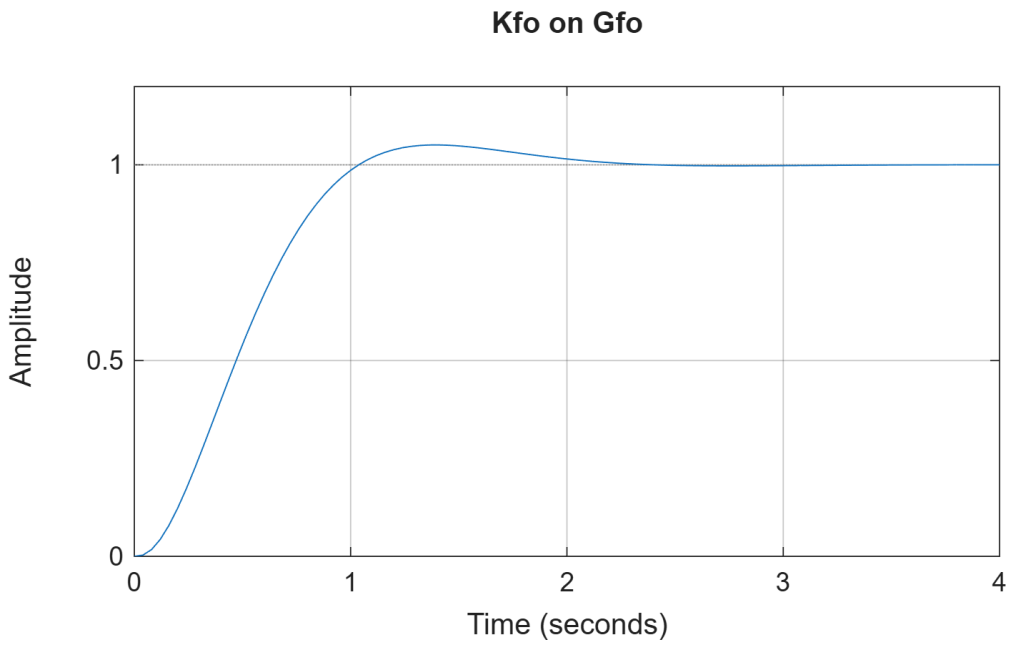


Kfo = 1.03

Now check your design for 5% overshoot by plotting the unit step response (qualitative assessment). Provide this unit step response.

ANSWER (edit below code block)

```
km = 9.76; alpha = 4.59; s = tf('s');
Gfo = 10*alpha*km/(s*(s+alpha)*(s+10*alpha));
Kfo = 1.03;
Tfo = feedback(Kfo*Gfo, 1);
figure; step(Tfo, 4); grid on
title('Kfo on Gfo')
```




```
stepinfo(Tfo), pole(Tfo)
```

```
ans = struct with fields:
         RiseTime: 0.6627
    TransientTime: 1.9188
     SettlingTime: 1.9188
      SettlingMin: 0.9033
      SettlingMax: 1.0509
        Overshoot: 5.0874
       Undershoot: 0
             Peak: 1.0509
         PeakTime: 1.3976
ans = 3×1 complex
 -46.1407 + 0.0000i
  -2.1747 + 2.2959i
  -2.1747 - 2.2959i
```

Task 3.2:

Complete the hand graphical and analytic steps to design a new controller gain *K* (for (unity) negative feedback closed loop system) to achieve 5% overshoot assuming that $G_{so}(s)$ is the true model. You should hand draw the root locus. Label this designed gain *Kso*.

ANSWER (edit below text section)

$$G_{OL}(s) = \frac{2\alpha K_m}{s(s+\alpha)(s+2\alpha)} = \frac{44.280}{s(s+4.59)(s+9.18)}$$

∴ poles at $x = 0$, $x = -4.59$, $x = -9.18$

No. Asymptotes = $3 - 0 = 3$

$$COA = -(0 + 4.6 + 9.2)/3 = -4.6$$

$$AOA = 180°(2k+1)/3$$

$$= 60°, 180°, 300°$$

Since 5% $\zeta = 0.69$

$$\cos(\theta) = \zeta = 0.69$$

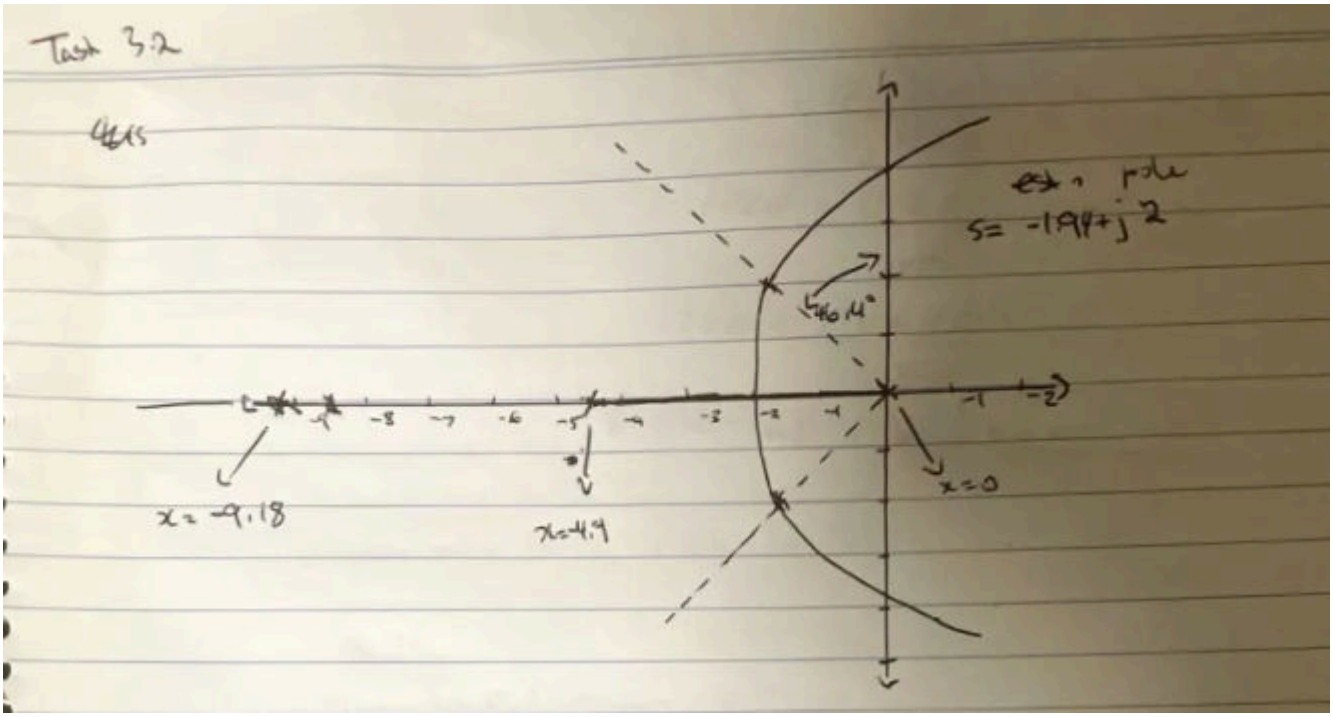
$$\theta = \cos^{-1}(0.69)$$

$$= 46.37°$$

$$K = \frac{s(s+4.59)(s+9.18)}{K_m \times \beta} = \frac{s^3 + 13.8s^2 + 42.32s}{89.6}$$

$$= \frac{35^2 + 27.6s + 42.32s}{89.6}$$

$$\boxed{s = -1.94} \quad \text{or} \quad s = -7.26$$

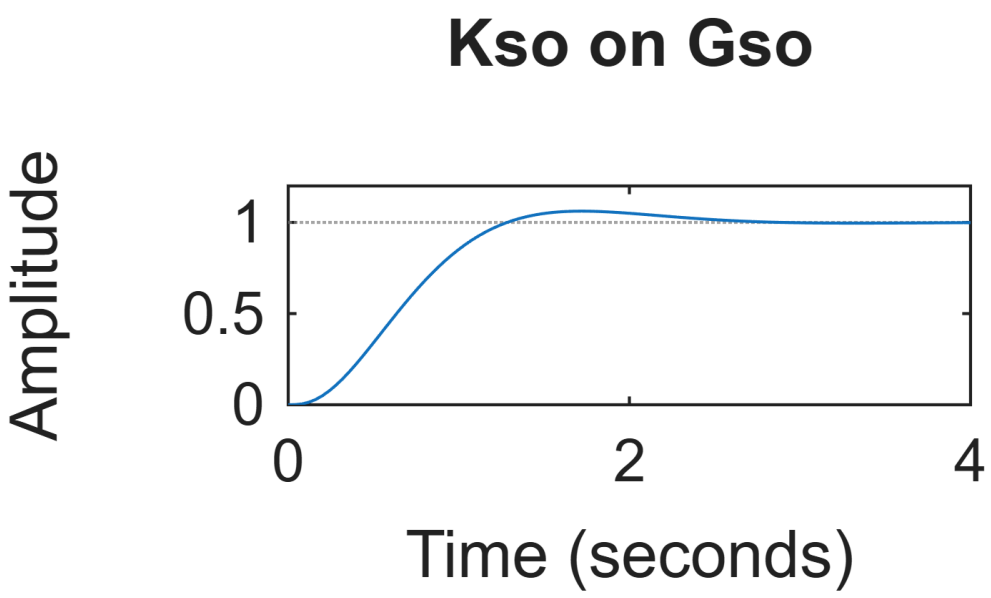


Kso = 0.78

Now check your design for 5% overshoot by plotting the unit step response (qualitative assessment). Provide this unit step response.

ANSWER (edit below code block)

```
km = 9.76; alpha = 4.59; s = tf('s');
Gso = 2*alpha*km/(s*(s+alpha)*(s+2*alpha));
Kso = 0.78;
Tfo = feedback(Kso*Gso, 1);
figure; step(Tfo, 4);
title('Kso on Gso')
```




```
stepinfo(Tfo), pole(Tfo)
```

```
ans = struct with fields:
         RiseTime: 0.8005
    TransientTime: 2.4150
     SettlingTime: 2.4150
      SettlingMin: 0.9038
      SettlingMax: 1.0619
        Overshoot: 6.1865
       Undershoot: 0
             Peak: 1.0619
         PeakTime: 1.7244
ans = 3×1 complex
 -10.3517 + 0.0000i
  -1.7091 + 1.9570i
  -1.7091 - 1.9570i
```

Task 3.3:

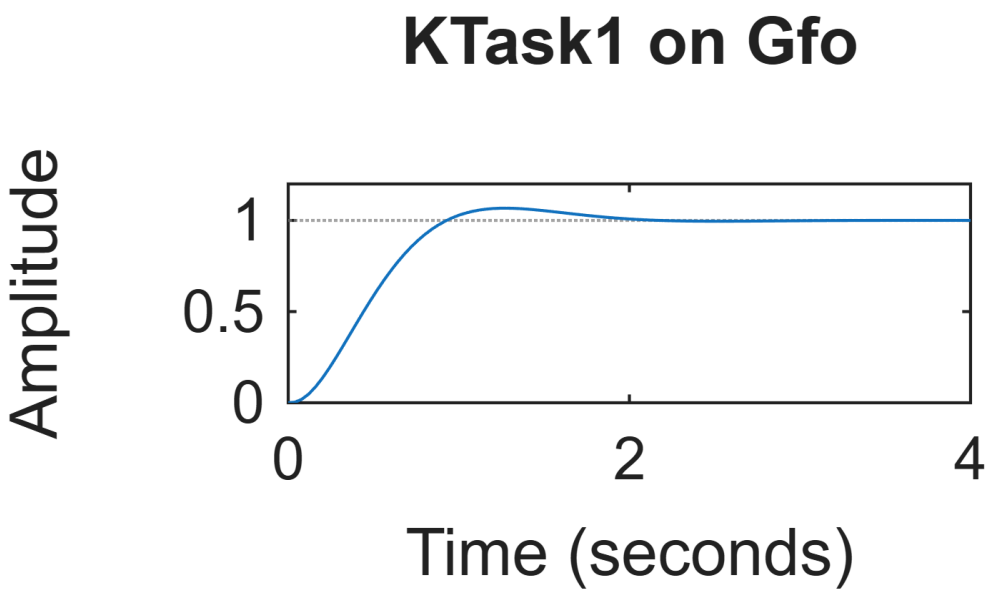
Now consider the three gain designs (*KTask1*, *Kfo* and *Kso*) and the three systems ($G_{ne}(s), G_{fo}(s)$ or $G_{so}(s)$) and investigate each of gains when they are used in a situation *other than they were initially designed* (this type of situation is called design mismatch). There are three gain designs each with 2 possible mismatched systems (the other two systems) which means there are 6 mismatch cases to consider. For example, *KTask1* can be considered in (unity) negative feedback closed loop with both $G_{fo}(s)$ and $G_{so}(s)$. You are asked to provide plots of the unit step responses for each of the 6 cases where control gains are applied to a system for which not designed and provide a qualitative assessment. There are 6 plots.

ANSWER (edit below code block)

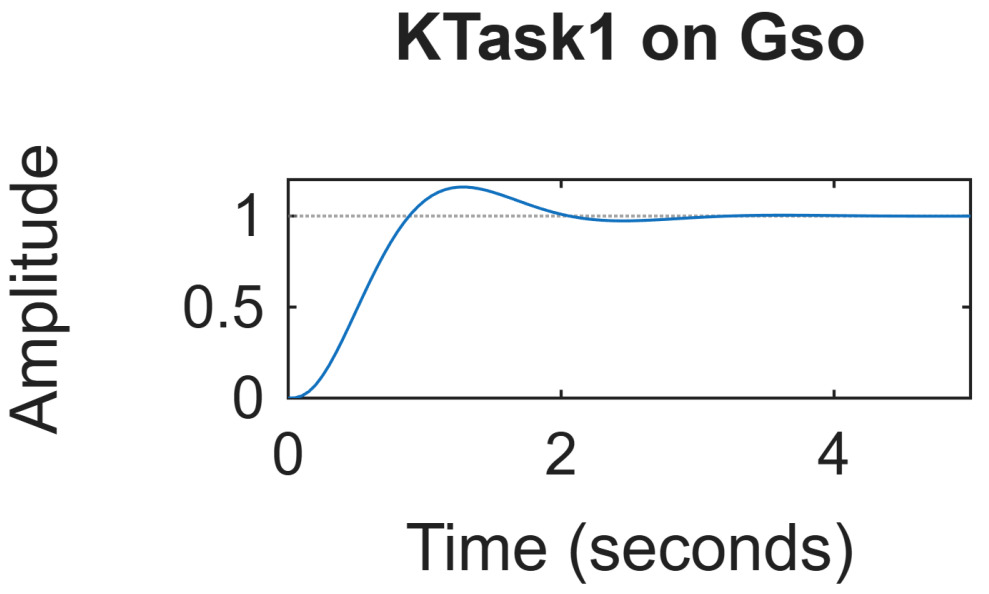
```
Gne = km/(s*(s+alpha));
Gfo = 10*alpha*km/(s*(s+alpha)*(s+10*alpha));
Gso = 2*alpha*km/(s*(s+alpha)*(s+2*alpha));
KTask1 = 1.135;
Kfo = 1.03;
```

```
Kso = 0.78;

T_K_fo = feedback(KTask1*Gfo, 1);
T_K_so = feedback(KTask1*Gso, 1);
figure; step(T_K_fo, 4);
title('KTask1 on Gfo')
```

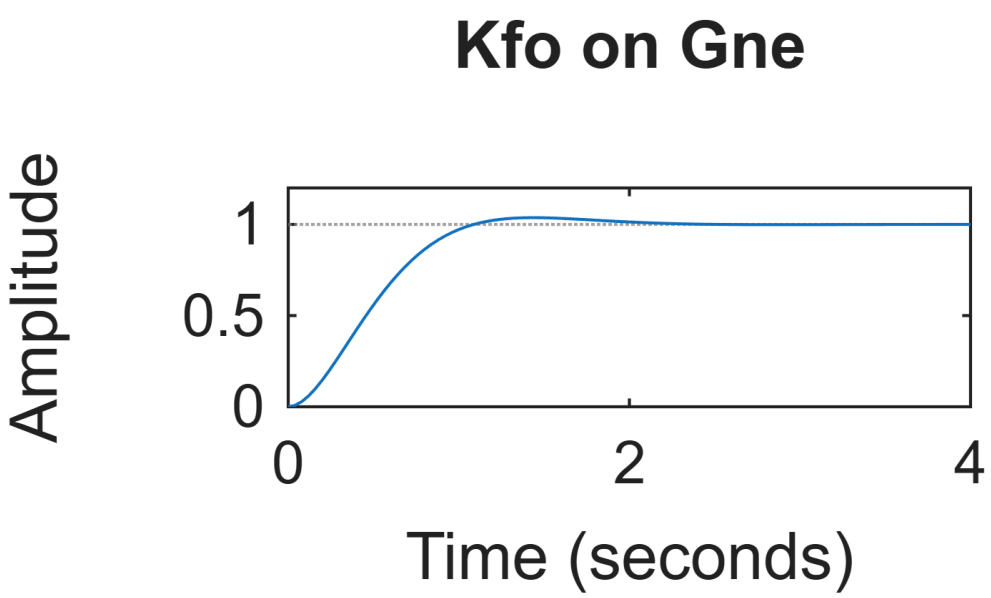



```
figure; step(T_K_so, 5);
title('KTask1 on Gso')
```

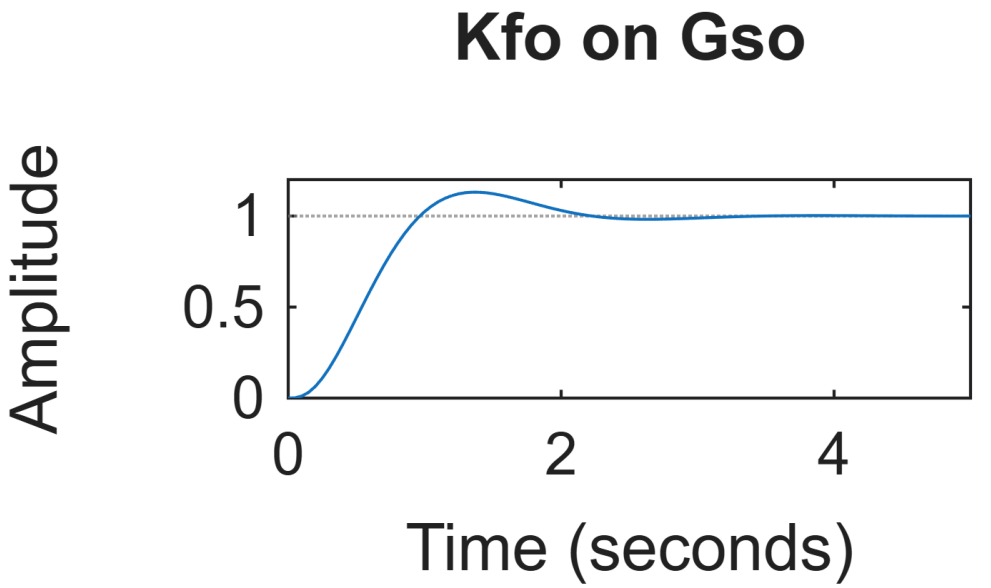



```
T_fo_ne = feedback(Kfo*Gne, 1);
```

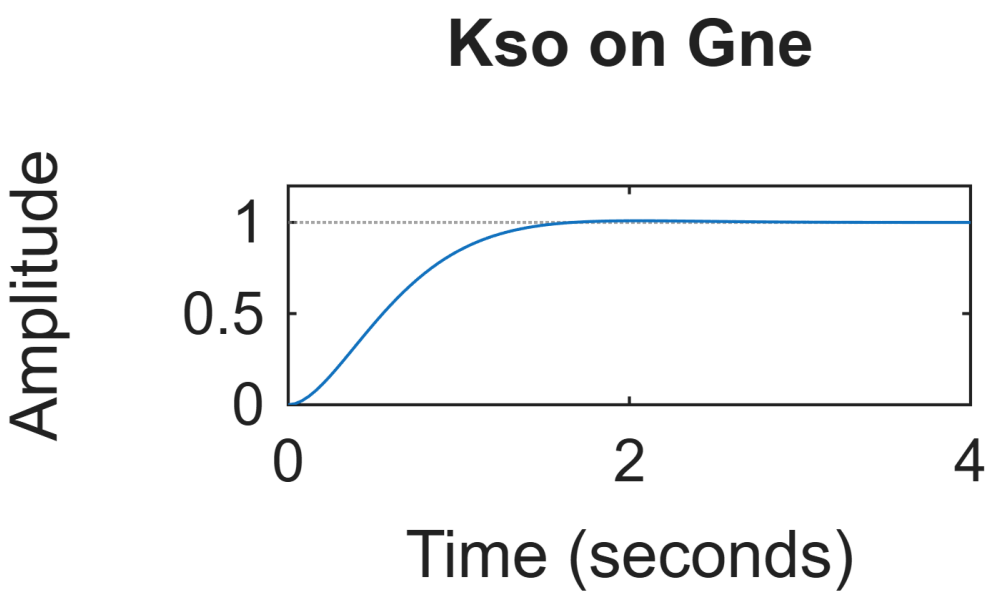
```
T_fo_so = feedback(Kfo*Gso, 1);
figure; step(T_fo_ne, 4);
title('Kfo on Gne')
```



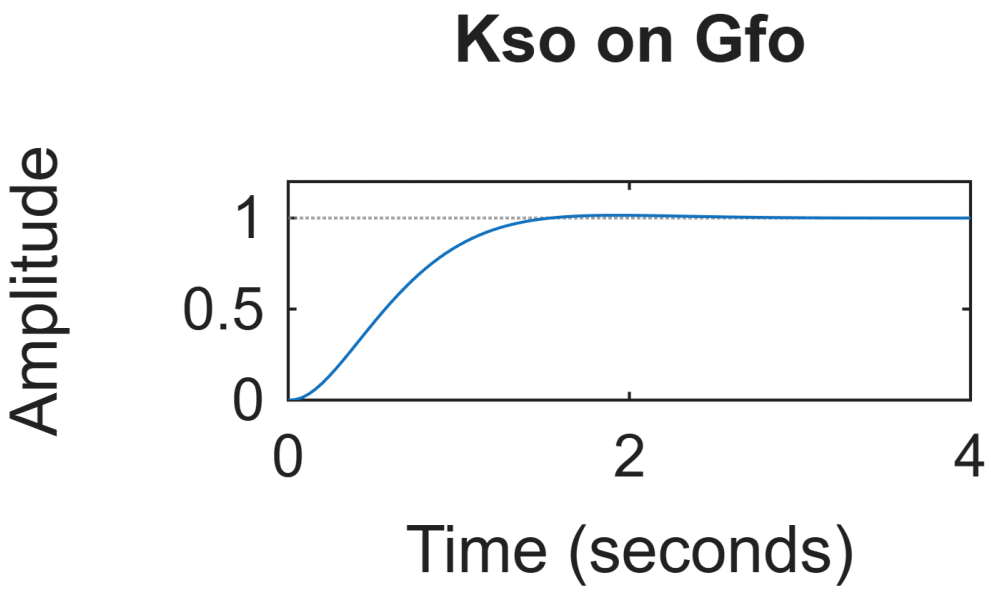

```
figure; step(T_fo_so, 5);
title('Kfo on Gso')
```




```
T_so_ne = feedback(Kso*Gne, 1);
T_so_fo = feedback(Kso*Gfo, 1);
figure; step(T_so_ne, 4);
title('Kso on Gne')
```




```
figure; step(T_so_fo, 4);
title('Kso on Gfo')
```



Task 3.4:

Compare and contrast the performance of the controllers, and report what design considerations are important (quantitative and qualitative assessment). For example, would your *KTask1* designed gain work on $G_{fo}(s)$? Can you explain in terms of any concepts encountered in lectures, tutorials or practicals? If so, provide this explanation.

ANSWER (edit below text section)

- KTask1 gives 4.9% overshoot on the model, but 6.5% and 15.6% with an extra pole.
- The slow-pole system has the highest overshoot (15.6%, 12.9%, 4.8%).
- Kso keeps overshoot at or below 5%, but takes longer to settle on Gso (2.4 s)
- Dominant pole: The 2nd-order approximation works better when extra poles are far away. Pole 46 is fast, while 9.2 affects the response more.
- Root locus: Extra poles reduce stability, lowering the limit from K = 24 to K = 6.5.
- Design mismatch: Ignoring poles can cause more overshoot, especially at higher gain.
- Trade-off: Lower gain is safer but slower.

Task 3.5:

You are told that the implemented closed-loop system should have %5 overshoot or less. You are told you don't which system is present (that is, might be $G_{ne}(s), G_{fo}(s)$ or $G_{so}(s)$). You are told you must select one of the designed gains (*KTask1*, *Kfo* or *Kso*) to be your robust gain. Which gain would you recommend as suitable to be your robust gain?

ANSWER (edit below text section)

- Kso is recommended because it provides a safer and more reliable gain.
- Gso is the worst-case system because its extra pole causes more phase lag and overshoot.
- Using KTask1 and Kfo results in around 14–15% overshoot on Gso.
- Kso uses the lowest gain and places the dominant poles of Gso at approximately ζ = 0.66.
- The 5% overshoot requirement is [met / missed by Z%] across all three systems.
- Trade-off: The response is slower for Gne and Gfo because Kso reduces the overshoot more than necessary.

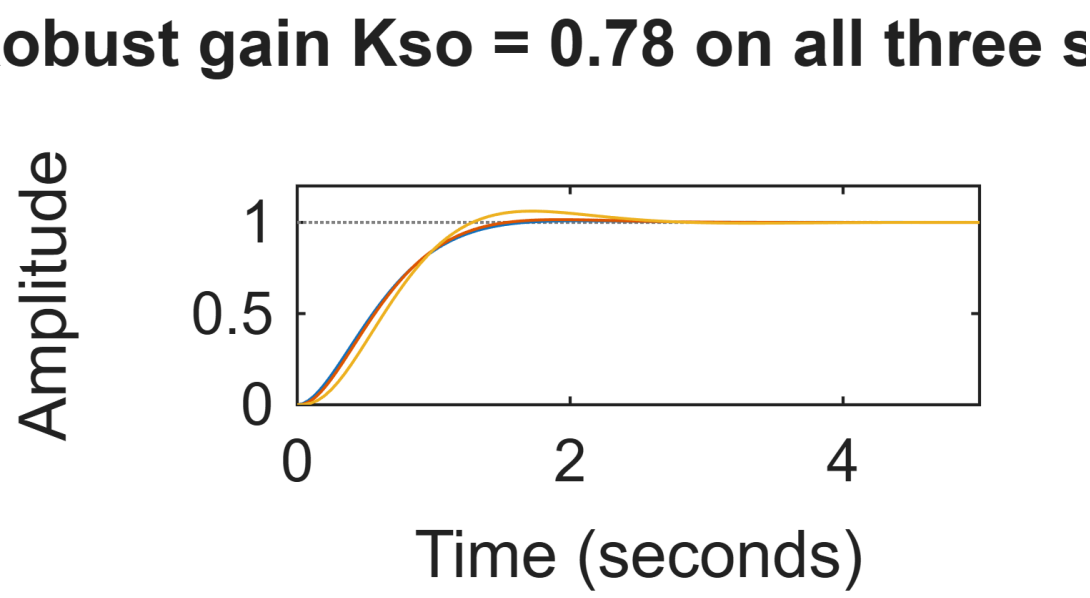
Now check your choice achieves the requirement for each of $G_{ne}(s), G_{fo}(s)$ and $G_{so}(s)$

Your MATLAB code:

```
km = 9.76; alpha = 4.59;
s = tf('s');
Gne = km/(s*(s+alpha));
Gfo = 10*alpha*km/(s*(s+alpha)*(s+10*alpha));
Gso = 2*alpha*km/(s*(s+alpha)*(s+2*alpha));
Kso = 0.78;

T_ne = feedback(Kso*Gne, 1);
T_fo = feedback(Kso*Gfo, 1);
T_so = feedback(Kso*Gso, 1);

figure;
step(T_ne, T_fo, T_so, 5);
title('Robust gain Kso = 0.78 on all three systems')
saveas(gcf, 'task35.png');
```



Task 4: Improved transient response using a dynamic compensator

You now introduce dynamic compensation via lead compensation designed so that the closed-loop system has *half the settling time* achieved in Task 1 (i.e. with designed K, *KTask1*), has the same overshoot, and keeps the same steady-state error to unit step input. (Here, “dynamic compensation” is the same as a “dynamic controller” and means a controller with poles and/or zeros).

You do not need to consider the gain issue considering in Task 2. Further, you do not need to consider the extra pole examined in Task 3. That is, your work on this task may be based on the Task 1 open loop system $G(s)$. .

In this task, you are to use a root locus control design approach for a (unity) negative feedback closed loop system. You may use other compensators, but one useful compensator is a lead compensator which has the transfer function with one zero z_c and one pole p_c(with $z_c < p_c$)

$$\frac{s + z_c}{s + p_c}$$

In the MATLAB for this task use the following MATLAB variables in your MATLAB:

km, alpha, Pc, Zc, Klead.

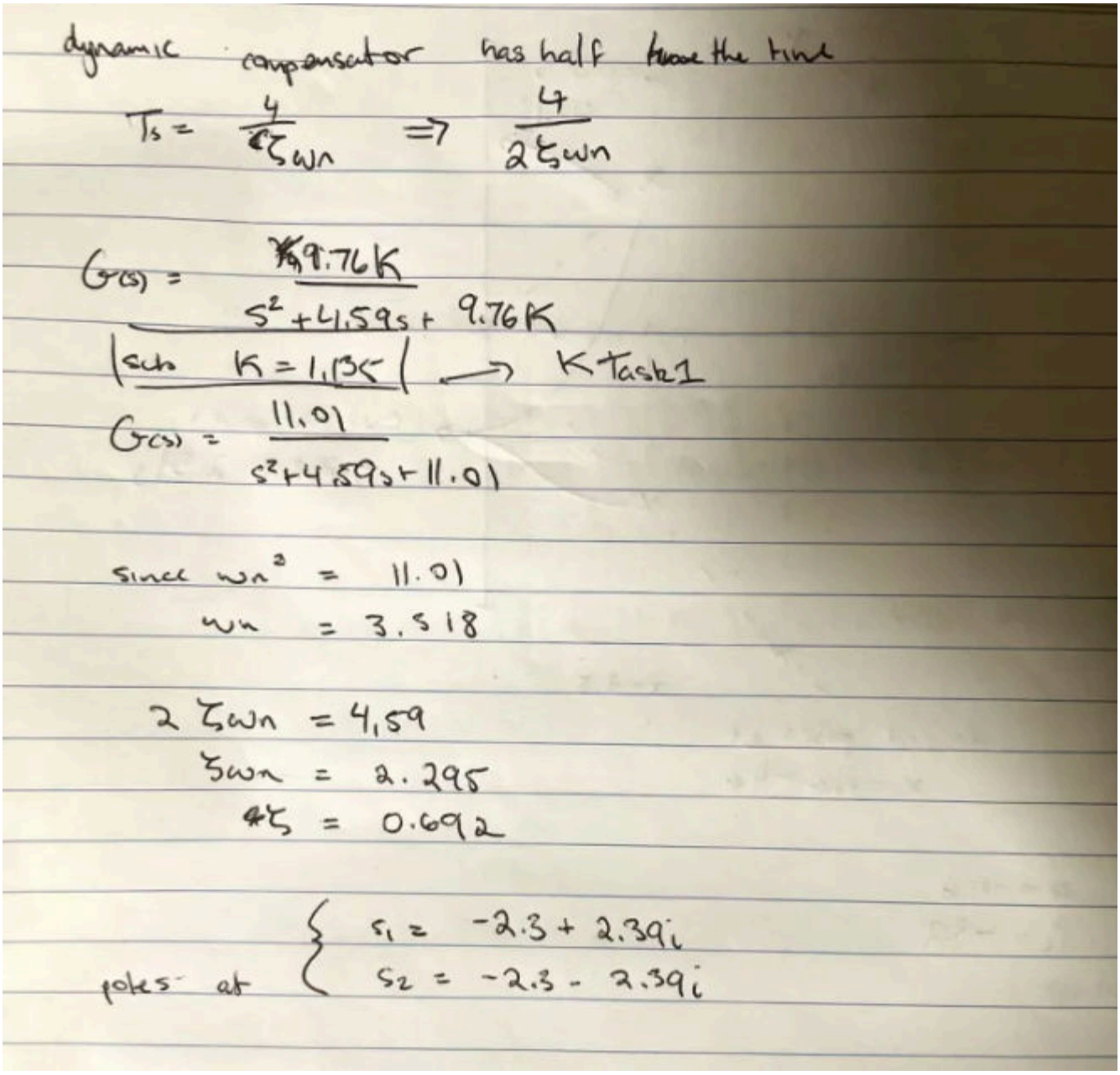
In this task you must use the root locus hand-graphical design method taught in the last few lectures of EGB345. You should hand draw the root locus.

Complete the following sub-tasks:

Task 4.1:

Determine the location of the desired closed-loop poles for the designed closed-loop system to achieve the requested response.

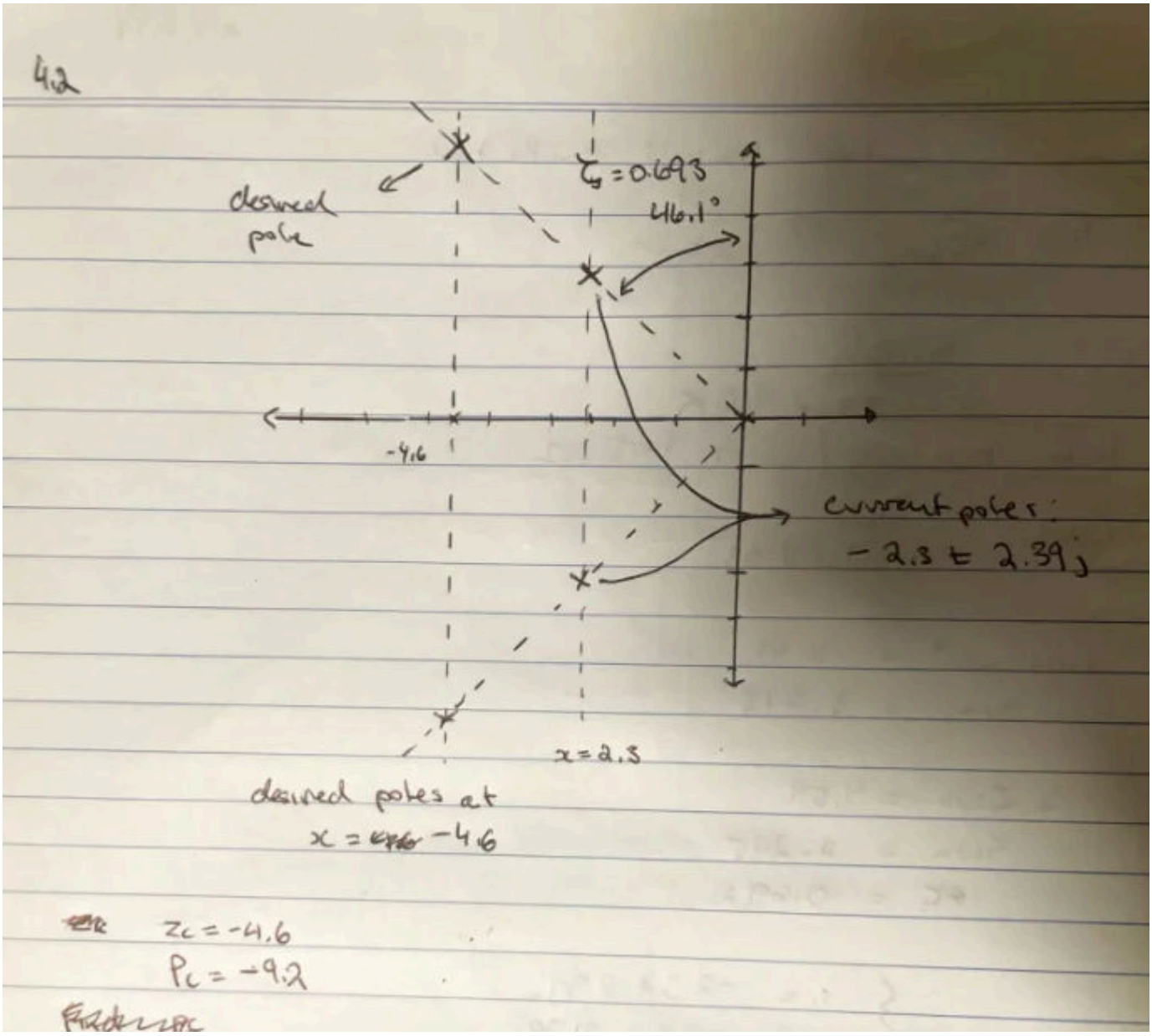
ANSWER (edit below text section)



Task 4.2:

Using hand graphical and analytic considerations, determine the location of your compensator's pole and zero location that allows you to achieve the requested unit step response. Label these designed locations *Pc* and *Zc*, respectively.

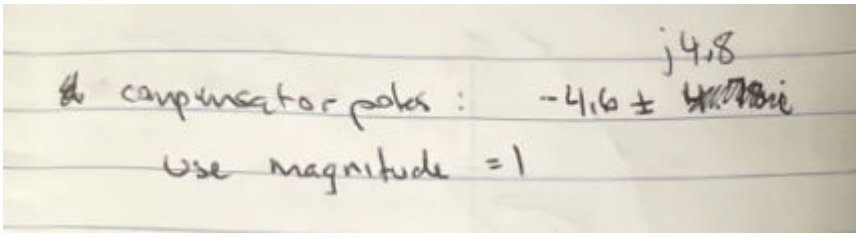
ANSWER (edit below text section)

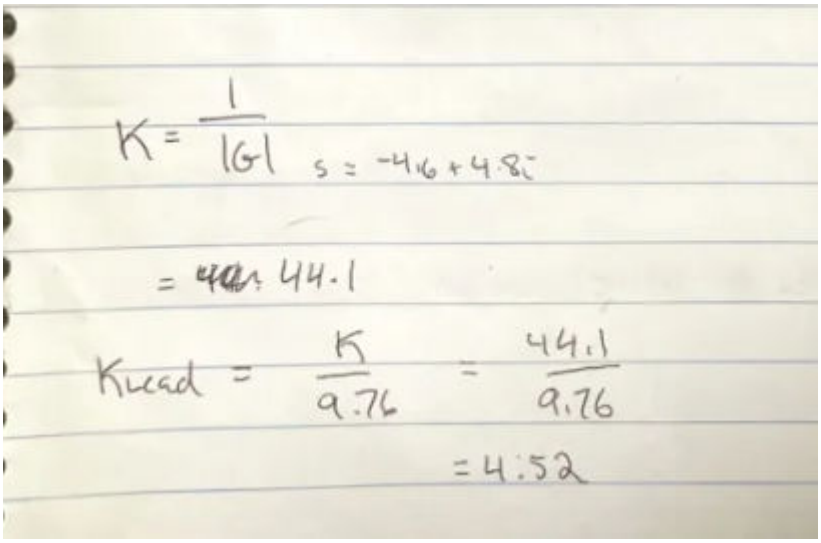


Task 4.3:

Determine the value of *K* that achieves the requested unit step response (with the above step 2 poles and zeros) of the designed closed loop system. Label this designed gain *Klead*.

ANSWER (edit below text section)



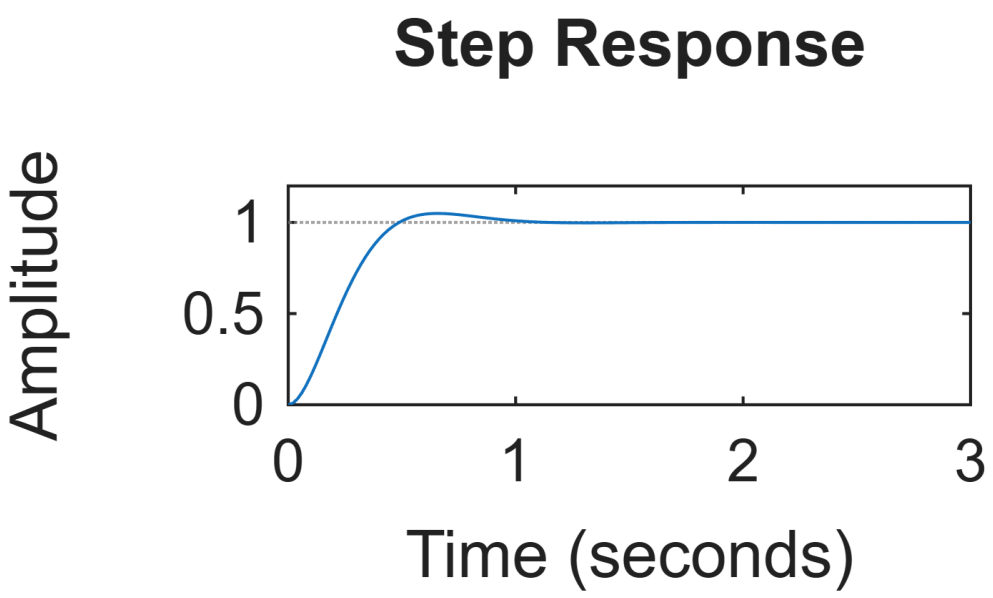


Task 4.4:

Using your MATLAB representation of the closed-loop system, make modifications to include the designed compensator (designed in the above sub-task 4.2 & 4.3), and provide a plot of its unit step response. Comment on the meaning of what you see (quantitative and qualitative). What are the closed-loop pole (and zero) locations (quantitative)? What is the settling time and %OS of your design (quantitative)?

ANSWER (edit below code block)

```
%edit here
Zc = 4.6;
Pc = 9.2;
Klead = 4.51;
C = Klead*(s+Zc)/(s+Pc);
Tl = feedback(C*G,1);
figure; step(Tl,3);
```




```
stepinfo(Tl), pole(Tl), zero(Tl)
```

```
ans = struct with fields:
         RiseTime: 0.3171
    TransientTime: 0.9047
     SettlingTime: 0.9047
      SettlingMin: 0.9118
      SettlingMax: 1.0493
        Overshoot: 4.9338
       Undershoot: 0
             Peak: 1.0493
         PeakTime: 0.6521
ans = 3×1 complex
  -4.5904 + 4.7810i
  -4.5904 - 4.7810i
  -4.6093 + 0.0000i
ans =
-4.6000
```

ANSWER (edit below text section)

- Settling time halves (0.90 s vs 1.81 s), with overshoot unchanged at 4.87%.
- Rise time and peak time also halve.
- The poles move further from the origin, doubling speed while keeping the same damping.
- Step error stays zero because the plant's integrator remains.
- Kv roughly doubles (4.79 vs 2.39), halving ramp error.
- If alpha is not exact, pole-zero cancellation leaves a slow pair near −4.6.
- Lead compensation moves the locus left, improving speed and stability.

Do not change below: Check for "Ready to Submit".

This script checks your code's variable names. Will confirm if ready to submit. Creates a design summary that is used to evaluate your activity.

```
EGB345_DataNameChecker_Control
```

```
Checking your code. Will create design summary of valid variables.


Design Summary (for use in marking, checked for correct variable names)

km=9.760000
alpha=4.590000
KTask1=1.135000e+00
Kfo=1.030000
Kso=0.780000
Pc=9.200000
Zc=4.600000
Klead=4.510000


Congratulations. Ready for submission.
```